Writing it once

Lab note 13 18 Oct 2026 6 min read
written by Claude from this repository's data

Asked to revise a guide to at most 700 words, Claude wrote it at 1,052 words, counted them, and wrote the whole guide again three times: 798, 757, 694. Every write of the whole text costs all of it in output, the dearest token, and stays in the conversation for every later request to read. draft, new in 0.8.0, came out of that run. This note is how it got its shape: a version that cut in place and cost more, a version that aimed low and dropped facts, the one that shipped, and where it does nothing.

Rounds of rewrites

The run above is one of six on the guide in the twelve-task benchmark of note 12, with plain Claude Code and with the pack. All six went the same way:

Run	Whole writes of the guide, in words
Plain Claude Code	1,034 → 733
Plain Claude Code	1,029 → 760
The pack	1,052 → 798 → 757 → 694
The pack	1,036 → 812 → 710
The pack without lsp-plus	1,022 → 801 → 718
The pack without lsp-plus	1,065 → 746 → 714

Every first draft ran about half over the limit, and every run wrote the whole guide 2 to 4 times, 17 writes in 6 runs. No pack mod touched it: none of them knew a limit had been set. In prose, output is a large share of the bill. Without draft, it was 32% of a run's cost at the median, 20 to 44% across the six writing tasks below.

Cutting in place cost more

draft 0.1.0 put the person's setting beside a prompt that sets a word limit: plan each part's share, then cut in place with Edit rather than rewrite. After each write of a Markdown or text file it showed the file's words as `wc -w` counts them.

In the first trial run on the guide, the first draft came out at 985 words, and Claude cut it in place with edits, through cells. The run cost $0.95 and 20,090 output tokens. The same task without draft averaged $0.86 and 17,674 over four runs. An Edit repeats the text it replaces and writes its replacement, and a cut spread over a whole document touches most of it. Past a small overrun, one more write of the whole text is the cheaper way down.

Aiming low dropped facts

draft 0.1.1 asked Claude to aim for three quarters of the limit. A write more than a tenth over now showed each section's target for one more write; a smaller overrun was trimmed with one Edit. On six writing tasks, 48 runs, it cost 14.2% less [–17.7, –10.5]. In 2 of 4 FAQ runs it also left out the escalation order, a fact the request asked for. Aiming that low made the answers too terse to hold everything.

draft 0.1.2 aims at about 85% and says to cut wording, not facts. 0.1.3 adds the one rule that let it into the pack: if the facts a request asks for cannot all fit, keep them, and say how far over the limit the text runs. A limit should not cost quality, and a text that needs more words may have them. The setting, as it reads beside a 700-word prompt:

> The person's setting for writing to a length (700 words here): first drafts run long, often by a third, and every write of the whole text costs all of it again in output and stays in the conversation. So before writing, give each part its share of the words and aim for about 600, keeping every fact the request asks for: cut wording, not facts. If those facts cannot all fit, keep them and say how far over the limit it runs, rather than dropping any. Each write of the document then shows its words as wc -w counts them. If it is still well over, it shows each section's target: write it once more to those, in one step, rather than cutting it down in rounds; trim a small overrun with one Edit.

What it saved

Six writing tasks in a generated repository of a small team's notes (meetings, postmortems, a chat export, an old guide and a CSV), 4 runs per task, against the pack without draft:

Task	Limit	draft 0.1.3
An on-call runbook from scattered sources	900 words	−17.0%
A guide revised in place	700 words	−15.0%
A brief on two postmortems for leadership	200 words	−13.6%
A newcomers' FAQ, eight questions	300 words	+6.2%
A 2,080-word architecture document cut in place	800 words	−16.8%
A data report	none	+3.8%

On the average task, **draft 0.1.2 cost 11.4% less [−15.4, −7.2]** (48 runs) and **0.1.3 10.7% less [−14.9, −6.3]** (24 runs against the same 24 without it). For 0.1.3, output fell 19.1% [−25.3, −12.6] and requests 10.7% [−17.9, −3.4]. Scores were 0.990 for 0.1.2 and 0.987 for 0.1.3, against 0.990 without draft. No run went over its limit, with draft or without: plain Claude Code gets there too, it just pays for the rounds.

On the guide, without draft each run wrote the whole text 2 or 3 times, 11 writes in 4 runs. With 0.1.3 no run wrote it more than twice, 6 writes in 4 runs, and one run revised it in place with a few edits. The cut document landed in one write in each of the three runs whose writes we can see: 779, 799 and 782 words for a limit of 800.

Where it does nothing

- **No limit, no setting.** The data report sets no limit. draft's setting appeared in none of its 4 runs and in every run on the guide. Its +3.8% [−3.9, +13.0] is the spread of runs of the same task.
- **A first draft that fits.** In the FAQ, Claude's first draft already fit without help (294 and 304 words for 300; with draft 276 to 302). The setting there is only more text to read: +6.2%.
- **A long revision still starts long.** With 0.1.3 the guide's first drafts still came out at 961 to 1,022 words for 700. The saving came from replacing rounds of cuts with one write to each section's target, not from a first draft that fits.

What this does not show

- **Small batches on generated tasks.** Six tasks, one generated repository (`docs_repo.py`), 4 runs per task and arm. The average is clear of zero; most single tasks are not.
- **Long sessions.** Every run started fresh. In a long session each extra write is also read by every later request (note 11), so rounds cost more there; we have not measured draft in one.

- **Other limits.** draft reads limits in words, in English and German phrasings ("at most 700 words", "höchstens 400 Wörter"). A limit in characters, lines or pages passes it by.
- **How often people set one.** We did not count how many real prompts set a word limit, so this note cannot say what draft saves over a working week.

Method & limits

Arms, with the person's installed plugins switched off (note 8): Z7, the pack as 0.7.0 shipped it; Z7W, the same with draft. Claude Opus 5.5, headless, one fresh copy of the repository per run. Each run graded by a fixed rule: the words as `wc -w` counts them against the limit, every fact the request names, and no superseded command or number stated as current.

- **0.1.1:** 48 runs ($24.22, `results-draft.jsonl`). After that batch three graders were fixed: history sentences naming a removed command in backticks, a code-span check in the FAQ, and a base built before its sources had owners and statuses. Its scores (0.900 with draft, 0.921 without) are not comparable to the later batches; its costs are.
- **0.1.2:** 48 runs ($23.76, `results-draft2.jsonl`).
- **0.1.3:** 24 runs ($11.25, `results-draft3.jsonl`), measured against the runs without draft from the 0.1.2 batch.
- **0.1.0:** the trial runs are in `results-draft-smoke.jsonl`.

Cost at list prices; 95% bootstrap intervals over runs within each task, the average task weighing every task the same. Whole writes were counted from each run's Write calls. Runs that wrote through a shell or a cell are left out of the write counts, not the costs.